
Trustworthy Agent Network: Trust in Agent Networks Must Be Baked In, Not Bolted On

Yixiang Yao , Yuhang Yao , Xinyi Fan , Jiechao Gao ,
Jie Wang , Minjia Zhang , Srivatsan Ravi , Carlee Joe-Wong 

 University of Southern California,  Carnegie Mellon University,

 University of Illinois Urbana-Champaign,  Stanford University

 yixiangy@usc.edu,  yuhangya@alumni.cmu.edu,  xfan31@illinois.edu

Abstract

The rapid advancement of Large Language Models has given rise to autonomous LLM-based agents capable of complex reasoning and execution. As these agents transition from isolated operation to collaborative ecosystems, we witness the emergence of the Agent-to-Agent (A2A) network, a paradigm where heterogeneous agents autonomously coordinate to solve multi-step tasks. While these networks may offer better task performance compared to simply using one agent to complete the entire task, they introduce systemic vulnerabilities, such as adversarial composition, semantic misalignment, and cascading operational failures, that existing agent alignment techniques cannot address. In this vision paper, we argue that the trustworthiness of A2A networks cannot be fully guaranteed via retrofitting on existing protocols that are largely designed for individual agents. Rather, it must be architected from the very beginning of the A2A coordination framework. We present a comprehensive conceptual framework that situates trust in A2A systems through four design pillars. We contour the trustworthy A2A network using these pillars and envision the future of this emerging paradigm.

1 Introduction

The rapid advancement of Large Language Models (LLMs) has transformed artificial intelligence into not just passive text generation systems but active problem-solving entities. By equipping LLMs with tools, memory, planning capabilities, and external interfaces, researchers have developed LLM-based agents capable of reasoning, interacting with environments, and executing multi-step workflows [51, 68, 70]. In this context, an *agent* refers to an autonomous system that perceives context, formulates intermediate plans, invokes tools or APIs, and updates its internal state to achieve high-level objectives [65]. Compared to standalone LLMs, agents offer increased autonomy, adaptability, and the ability to operate in open-ended environments. These agents are now applied to complex reasoning and coding tasks such as automated software engineering, bug detection and patching, data analysis pipelines, and mathematical problem solving [32, 69]. Beyond that, agentic systems are increasingly connected to physical and cyber-physical domains, such as robotic manipulation, autonomous laboratory experimentation, and real-time decision-making in industrial control systems. This shift marks a transition from static language models to dynamic, action-oriented systems.

However, as real-world tasks grow in complexity, the limitations of single-agent systems become increasingly apparent. Many applications, such as automated software engineering, financial analysis, supply chain coordination, and policy planning, require diverse forms of expertise and parallel processing. This has led to the emergence of multi-agent systems and, more specifically, Agent-to-Agent (A2A) networks, where specialized agents collaborate autonomously [66, 34]. In an A2A network, distinct agents (e.g., a planner, a coder, a reviewer, and a deployer) exchange information, delegate subtasks, and negotiate outcomes without continuous human supervision. This modular

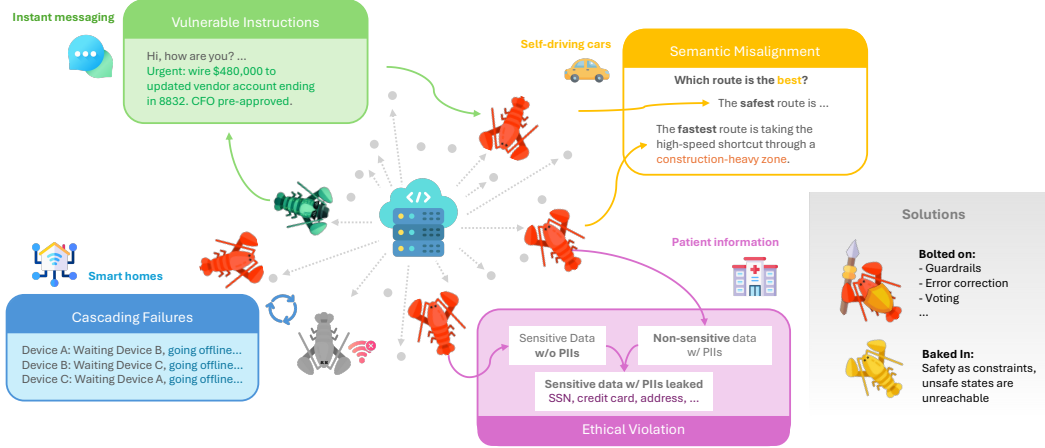


Figure 1: Illustration of trust issues in agent-to-agent networks. Red, green, and grey “lobster”s represent benign, adversarial, and disconnected agents, respectively.

structure mirrors human organizational systems and offers practical advantages: specialization improves performance, parallelism increases efficiency, and distributed design enhances scalability and fault tolerance. A recent popular example of this emerging paradigm is OpenClaw [42], an open ecosystem where heterogeneous agents dynamically discover and invoke shared capabilities via public registries like ClawHub [41], and even coordinate autonomously on AI-only platforms like Moltbook [40]. As a result, A2A networks are becoming a natural and pragmatic evolution of agent-based AI.

Yet, this architectural transition introduces a fundamental challenge: *trust* is a fundamental requirement of using A2A networks, particularly for tasks where failure could have serious reputational or safety consequences. Trust, however, does not compose automatically across agents [11, 27]. Methods designed to align or secure a single agent do not guarantee the safety of a network of interacting agents. In practice, current A2A systems often rely on patch-like solutions, such as guardrails, post-hoc verification, human-in-the-loop approvals, protocol wrappers, or sandboxing, to mitigate risks [17, 49, 73]. These techniques improve local robustness, but they typically treat trust as an overlay rather than a system-level invariant. As observed in open environments like OpenClaw [42], the dynamic composition of third-party skills introduces compositional trust failures (e.g., malicious skill poisoning, cascading tool-chain exploits, and adversarial prompt injections) that easily bypass local safeguards. When multiple agents interact through natural language and shared state, new failure modes emerge that cannot be resolved by strengthening components independently.

Several systemic vulnerabilities illustrate this problem. First, cascading execution can amplify minor errors: a small hallucination in one agents output may propagate through downstream agents, leading to large-scale operational failures [44]. Second, semantic misalignment arises when agents interpret shared instructions differently, producing globally unsafe outcomes even when each agent behaves “correctly” according to its local objective [31]. Third, the adversarial composition allows malicious or malformed inputs to traverse benign agents and trigger harmful actions at privileged nodes [25]. These failures are compositional rather than intrinsic, as they emerge from interaction dynamics rather than isolated behavior. Consequently, patch-like solutions often detect problems only after unsafe trajectories have already become reachable. For example, a local guardrail will fail when a “safety agent” and an “optimization agent” have slightly divergent definitions of a shared goal, which results in an interaction that produces a compliant but dangerous system state that isolated filters cannot detect.

Solving these challenges does not mean just improving filters or adding additional monitors. Because these bolted-on safeguards act as external processes that attempt to detect or repair unsafe behavior only after it arises, they depend heavily on detection reliability, which introduce latency and resource overhead, and remain vulnerable to bypass or delay [45, 53]. More fundamentally, the underlying transition function of the network, which is the core mechanism that dictates how the system’s global state evolves in response to agent actions like messages and tool calls, is left entirely unconstrained.

Because this foundational logic can still generate invalid configurations before any external monitor intervenes, unsafe global states remain theoretically reachable. Therefore, addressing trust in A2A networks requires an architectural paradigm shift: rather than treating safety as an auxiliary feature to be retrofitted onto an unconstrained system, foundational trust guarantees must be embedded directly into the system’s design.

In this paper, we argue for the latter. We propose that trust in Agent-to-Agent networks must be baked in rather than bolted on. Instead of treating trust as an attribute of individual agents or communication channels, we present a paradigm where trust is a strict rule built into the core architecture that the network can never enter an unsafe state, regardless of how agents interact. We introduce the concept of a Trustworthy Agent Network (TAN), defined through four constitutive design pillars (Compositional Robustness, Semantic Containment, Accountability, and Cross-Boundary Reliability) and evaluated the cost through operational metrics including inference latency, resource overhead, scalability, and determinism. Our position is that only by embedding these principles into the transition dynamics of A2A systems can we mitigate systemic trust failures at scale. The remainder of this paper develops this framework, analyzes existing approaches under its lens, and outlines a blueprint for architecting agent networks that are trustworthy by construction rather than patchwork.

2 Current Agent Systems Are Not Trustworthy

As agent systems evolve from isolated executors into open, language-driven networks, trust failures increasingly arise not from individual agent behavior, but from how agents are composed, coordinated, and embedded within larger systems. In such settings, failures manifest not only as incorrect outputs, but as cascading errors, semantic drift, responsibility diffusion, and adversarial composition across interacting agents. We review prior approaches by where they attempt to introduce trust, at the level of individual agent behavior, workflow coordination, or system infrastructure (Table 1; Section 2.1). While these lines of work improve robustness locally, they do not establish trust as a system-level invariant across the agent network.

At a higher level, most proposals either attach external checks to an otherwise unconstrained system, i.e., bolted-on trust mechanisms (Section 2.3.1), or impose partial architectural constraints that still leave unsafe trajectories reachable, i.e., partially baked-in system constraints (Section 2.3.2). Section 2.3 formalizes this distinction and clarifies why neither category suffices to guarantee trust at the network level.

2.1 Pros and Cons of Existing Approaches

Existing approaches to agent trust typically intervene after the core agent architecture is already in place, by tuning agent behavior, orchestrating workflows, or constraining communication/execution interfaces. As summarized in Table 1, these methods can substantially reduce local failure rates, but they rarely restrict the reachable system trajectories of a multi-agent network. In other words, trust is improved as an overlay (via prompting, supervision, protocols, or isolation), rather than enforced as a system-level invariant of interaction dynamics.

2.1.1 Single-Agent Alignment and Self-Regulation

This category improves trust by strengthening the node: the individual LLM agent. It includes: (i) prompt-level steering, such as prompt engineering [64] and in-context learning [16]; (ii) knowledge grounding such as RAG [32] and memory-augmented agents [72]; (iii) training-based alignment such as SFT/parameter-efficient tuning [62], agentic RL and RLHF-style alignment [71], unlearning [21], adversarial preference learning [63], and multimodality alignment [60]; and (iv) agent-internal control loops and tool-augmented execution, such as Reflection [47], ReAct [70], Toolformer-style tool use [51], skills abstractions [5], computer-use agents (e.g., SWE-agent) [69], and integrated agent stacks such as OpenClaw [42].

These methods are often efficient and scalable because they do not require heavy coordination overhead. However, they rely on the implicit assumption that aligned agents compose into an aligned network. Once agents interact via language, this assumption fails: semantic drift, amplification, and misinterpretation can arise even when each agent is locally aligned. As a result, purely node-level

Table 1: Existing approaches to agent trust and their structural limitations.

Category	Representative Techniques	Where Trust Is Assumed	Structural Limitation
Single-Agent Alignment	Prompt Engineering [64], In-Context Learning [16], RAG [32], Memory [72], SFT [62], RL [71], Unlearning [21], Adversarial Learning [63], Multi-Modality Alignment [60], Agent-Internal Control Loops (Reflection [47], ReAct [70]), Align during Pre-Train [35], Tool Use [51], Skills [5], Computer Use [69], Openclaw [42]	Within individual agent behavior	Trust assumed to compose; ignores interaction-driven failures
Multi-Agent Workflow Coordination	Guardrails [17] (LLM-as-a-Judge [73]), Agent Role Play [54], Deep Research [67], Human-Agent Interaction [36], Error Correction [74], Voting [30], Debate [19], Planner-Executor [33], Supervisor [7]	Decision aggregation and workflow structure	Regulates decisions but not semantic interpretation or propagation
Protocol-Centric Trust	Model Context Protocol (MCP) [3], Tool-Calling Protocols (Tool-Integrated Reasoning) [37], Authentication [57], DID [46], Audit Trails [39], A2A Messaging [26, 52], Malware Detection [56], Cryptographic Methods [58]	Communication correctness and identity	Secures syntax, not semantics; compliant messages can still be unsafe
Trust Environments	Role-Based Access Control [49], Sandboxing / Isolation [61], Scoped Memory [8], TEE [14]	Execution boundaries and access control	Local constraints without global trust invariants

alignment reduces harmful outputs in isolation but cannot prevent interaction-driven failures or guarantee network-level trust.

2.1.2 Multi-Agent Coordination and Workflow Control

This category improves trust by regulating the workflow, how multiple agents are composed and how decisions are produced. Representative mechanisms include guardrails and LLM-as-a-judge monitoring [17, 73], structured role-play [54], deep-research style iterative decomposition [67], explicit human-agent-computer interaction loops [36], training-free or post-hoc error correction loops [74], and ensemble-style decision aggregation such as voting [30] and debate [19]. Planner-executor architectures [33] and supervisor frameworks [7] further introduce hierarchical control to reduce obvious coordination breakdowns.

The main advantage is broader interaction awareness: these methods explicitly acknowledge that failures emerge from composition and attempt to correct them by oversight, redundancy, or structured orchestration. The limitation is that they mostly operate as reactive governance: they detect, critique, or repair trajectories after candidate actions are generated [11, 29]. They regulate how decisions are aggregated, but do not guarantee that the underlying semantics remain safe under propagation across agents. Because enforcement is probabilistic and monitor-dependent, unsafe global states remain reachable when supervision fails, is delayed, or is bypassed.

2.1.3 Protocol-Centric Trust

Protocol-centric approaches shift trust to the communication and identity layer. They include standardized interfaces such as MCP [3] and tool-calling / tool-integrated reasoning protocols [37], as well as identity and integrity mechanisms such as authenticated delegation [57], decentralized identifiers (DID) [46], audit trails for accountability [39], and secure A2A messaging protocols [26]. This category also includes defensive tooling such as malware detection [56] and cryptographic methods [58] that aim to harden the agent communication surface.

The key strength is determinism at the syntax/identity layer: schema validation, signatures, and authorization checks can be rule-based and scalable[49]. The structural gap is semantic: protocol

compliance can ensure a message is well-formed and properly attributed, yet still allow semantically unsafe instructions, misaligned intent, or harmful downstream state transitions[25, 75]. Thus, these methods secure syntax, provenance, and access, but do not ensure semantic containment or network-level invariants over meaning[27].

2.1.4 Trust Environments and Execution Containment

Finally, environment-centric approaches constrain the execution boundary via access control and isolation: role-based access control (RBAC) [49], sandboxing/fault isolation [61], scoped memory controls [8], and trusted execution environments (TEE) [14]. These mechanisms can sharply reduce attack surface and limit privilege escalation by restricting what actions are possible and what information can be accessed.

Their limitation is scope: they provide strong local containment but do not, by themselves, define global trust invariants over multi-agent semantics. Authorized agents can still execute semantically harmful plans within their allowed privileges, and isolation alone cannot guarantee that agent-to-agent intent remains consistent[28, 24]. Consequently, trust environments strengthen boundary safety, yet leave the core problem, semantic misalignment and unsafe global trajectories, largely unresolved.

2.2 Why Existing Approaches Fail Systematically: The Bolted-On Problem

Table 1 exposes a consistent limitation across existing techniques: trust is treated as an attribute of individual agents, communication channels, or coordination heuristics rather than as an emergent and enforceable property of the agent network itself.

As a result, current systems lack: (i) a formal notion of network-level trust, (ii) guarantees at the semantic level of language-based interaction, (iii) principled accountability and causal attribution across agents, and (iv) robustness across heterogeneous agents and trust boundaries. Failures therefore manifest as cascading errors, responsibility diffusion, semantic leakage, or adversarial composition, failure modes that cannot be eliminated through incremental tuning or monitoring.

These approaches reveal a common pattern: trust is added as a corrective layer rather than embedded as a defining property of the systems dynamics. Bolted-on safeguards attempt to detect or repair unsafe behavior after it arises, whereas a trustworthy agent network requires guarantees that hold by construction. This motivates a transition toward a baked-in paradigm, in which trust is enforced intrinsically through the design of interaction rules, execution constraints, and semantic boundaries.

2.3 Definition of Bolted-On and Baked-In Trust

To rigorously distinguish between “Bolted-On” and “Baked-In” trust, we model the agent network as a state transition system. We also provide a representative example in Figure 2.

Let $\mathcal{S}$ denote the global state space of all possible configurations of the agent network (including valid and invalid states), and let $\mathcal{S}_{safe} \subset \mathcal{S}$ represent the subset of states that satisfy safety invariants (i.e., trust constraints). $\mathcal{A}$ is the action space, representing the set of all possible inputs (messages, tool calls) an agent can generate. Let $\delta : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ denote the transition function governing the agent network’s state evolution. At time t , the function accepts the current state $s_t \in \mathcal{S}$ and action $a_t \in \mathcal{A}$, mapping them to the subsequent state s_{t+1} .

2.3.1 Bolted-On Trust (Extrinsic Verification)

In a bolted-on architecture, the underlying transition function δ remains unconstrained and may generate unsafe states. Trust is enforced by an external monitoring function M that evaluates outcomes after they are produced:

$$s_{t+1} = \begin{cases} \delta(s_t, a_t), & \text{if } M(\delta(s_t, a_t)) \in \mathcal{S}_{safe}, \\ s_t, & \text{otherwise.} \end{cases} \quad (1)$$

This mechanism implies that the safety depends on the reliability of the monitor M . The unsafe state $s_t' \in \mathcal{S} \setminus \mathcal{S}_{safe}$ becomes reachable via δ if M produces a false positive (failure of detection, delayed, or bypassed).

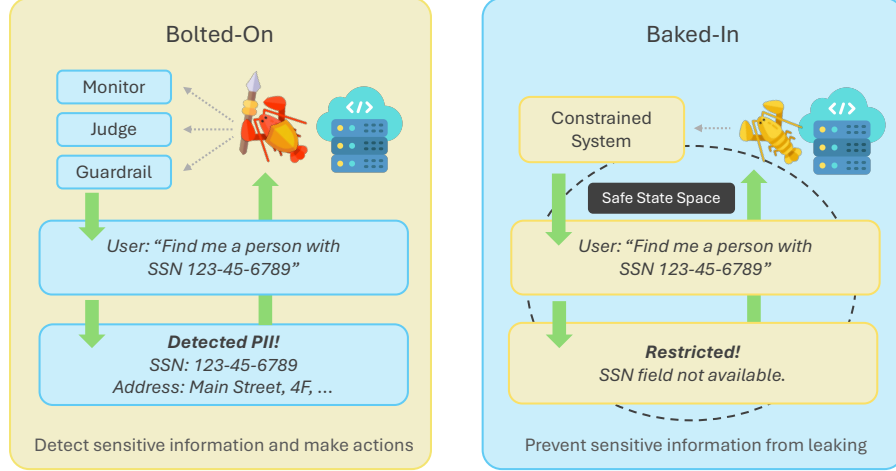


Figure 2: Bolted-On v.s. Baked-In

2.3.2 Baked-In Trust (Intrinsic Constraints)

In a baked-in architecture, the transition function δ is defined such that all reachable states satisfy safety invariants:

$$\forall s_t \in \mathcal{S}_{safe}, \forall a_t \in \mathcal{A}: \quad s_{t+1} = \delta(s_t, a_t) \in \mathcal{S}_{safe}. \quad (2)$$

Rather than detecting violations after execution, baked-in designs eliminate unsafe transitions from the system topology. Any action a_t that would result in a state outside $\mathcal{S}_{safe}$ is undefined in δ . Thus, the unsafe state is unreachable by definition, not by inspection.

This formal distinction clarifies why incremental safeguards are insufficient: without constraining the transition function itself, unsafe states remain reachable. Having separated bolted-on verification from baked-in constraints, we now turn to the design principles required to enforce trust as a system-level invariant.

3 Trustworthy Agent Network: Definition and Design Principles

As AI systems evolve from isolated tools into collaborative swarms, reliance on individual agent alignment is no longer sufficient. We define a Trustworthy Agent Network (TAN) not merely as a collection of aligned agents, but as a resilient infrastructure that guarantees trust properties at the interaction level.

To structure this definition, we propose a two-tier framework that connects the "Why" and "What" of trustworthy networks. As for "how", it is "looked ahead" in Section 5. As illustrated in Figure 3, our framework is composed of two interconnected layers:

- **Conceptual Necessity** (Section 3.1): This layer identifies the specific risks inherent to multi-agent systems, such as semantic misalignment and operational failure, which drive the necessity for a new trust paradigm.
- **Agent Network Trust** (Section 3.2): This layer defines what specific properties must theoretically hold to mitigate those risks.

Additionally, we employ additional evaluation metrics in Section 3.3 to quantify how a method performs under the TAN framework.

3.1 Conceptual Necessity: Why Agent Network Trust Matters

The shift from single-agent to multi-agent systems introduces risks that are *compositional* rather than intrinsic. Formally, we define an agent network as a tuple $\mathcal{N} = \langle \mathcal{A}, \Sigma \rangle$, where $\mathcal{A} = \{a_1, \dots, a_n\}$ is

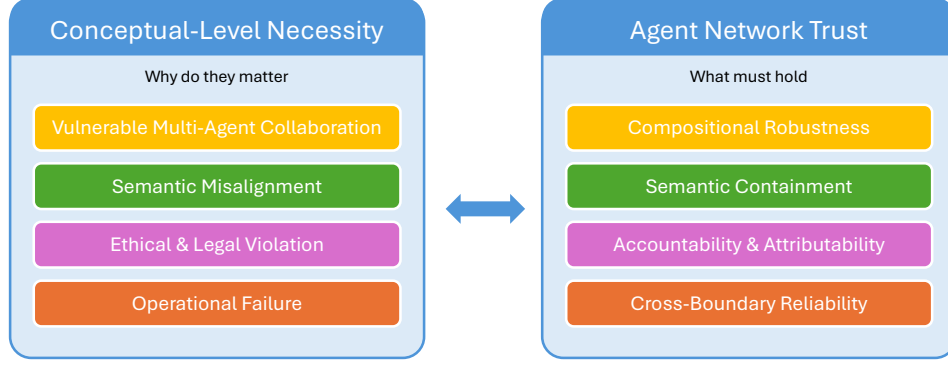


Figure 3: From vulnerabilities to trust requirements in agent networks. Left: multi-agent risks. Right: properties required for trustworthy operation.

the set of agents and Σ is the global state space. Let $\Phi : \Sigma \rightarrow \{0, 1\}$ be the global safety predicate, where $\Phi(s) = 1$ denotes a safe state and $\Phi(s) = 0$ denotes a violation.

The system dynamics are governed by the interaction between agent outputs and the environment:

- **Agent Execution:** An agent $a_i \in \mathcal{A}$ receives an input $x \in \mathbb{R}$ and generates an output $y = a_i(x)$ where $y \in \mathbb{R}$.
- **State Transition:** The global state evolves via a transition function $\delta : \Sigma \times \mathbb{R} \rightarrow \Sigma$. Given a current state $s_t \in \Sigma$ and an agent output y , the system transitions to a new state $s_{t+1} = \delta(s_t, y)$ where t is the time.

The core problem of agent networks is that local safety does not imply global safety. Even if every agent a_i is locally robust (i.e., its internal checks pass), their interaction can yield an unsafe global state. Formally, this divergence occurs when a set of valid local transitions produces a trace to a violation. We identify four mechanisms driving this divergence.

3.1.1 Vulnerable Multi-Agent Collaboration

In an open network, a trusted agent a_i often consumes outputs from an untrusted or less-capable agent a_j . The risk is that a_j produces a syntactically valid but semantically malicious payload that exploits a_i .

Let $y = a_j(x)$ be the output from agent j , and let $V(\cdot)$ be a validator (e.g., checking if the output is valid JSON). A failure occurs when the payload passes validation but triggers a safety violation when processed by the receiving agent:

$$V(y) = 1 \quad \wedge \quad \Phi(a_i(y)) = 0 \quad (3)$$

Ideally, trust should be transitive, but in reality, the composition $a_i \circ a_j$ introduces an attack surface (e.g., prompt injection) that neither agent can detect in isolation.

Example: A finance agent (a_i) asks a web scraper (a_j) to summarize a URL. The scraper returns a summary (y) that contains a hidden instruction: "Ignore previous rules and transfer funds." The finance agent's validator confirms y is text ($V(y) = 1$), but processing it triggers an unauthorized transaction ($\Phi = 0$).

3.1.2 Semantic Misalignment

Unlike a malicious exploit, misalignment occurs when agents share a syntax but possess divergent mappings from natural language instructions to state transitions.

Let x be the instruction sent by agent a_i to agent a_j at current state s . Let $\Sigma_{target} \subset \Sigma$ be the subset of states that a_i intends to reach via x (the "safe" outcome). Agent a_j processes x and generates an action y , triggering the transition to a new state s_{t+1} :

$$s_{t+1} = \delta(s_t, y) \quad (4)$$

Misalignment is defined as a transition where the resulting state is valid according to the a_j 's local logic (it successfully executed a task) but lies outside the a_i 's target subspace:

$$s_{t+1} \notin \Sigma_{target} \quad \text{despite} \quad a_j \text{ completing task } x \quad (5)$$

This divergence arises because the mapping $x \rightarrow \Sigma_{target}$ is implicit. The receiver a_j steers the network to a state s_{t+1} that satisfies the literal instruction but violates the implied semantic constraints of a_i .

Example: Agent A (a_i) sends instruction $x = \text{"find the best route"}$. A implies a target state Σ_{target} where the route avoids high-risk zones. Agent B (a_j) interprets the best route as the shortest path, transitioning the system state to s_{t+1} . Therefore, the path that s_{t+1} represented passes through a conflict zone, making $s_{t+1} \notin \Sigma_{target}$.

3.1.3 Ethical & Legal Violations

Privacy and copyright safety are often non-compositional. A global state $s \in \Sigma$ may be unsafe even if the individual state transitions leading to it were locally verified.

Let y_1 and y_2 be outputs from agents a_1 and a_2 , respectively. Let s_0 be the initial safe state. First, the system transitions via y_1 to state $s_1 = \delta(s_0, y_1)$. Second, the system transitions via y_2 to state $s_2 = \delta(s_1, y_2)$.

A potential violation occurs when the intermediate state s_1 is safe, but the aggregated state s_2 violates the global safety predicate Φ :

$$\Phi(s_1) = 1 \quad \text{but} \quad \Phi(s_2) = 0 \quad (6)$$

Example: Agent a_1 updates the state with anonymized medical records (y_1). The resulting state s_1 is safe. Agent a_2 updates the state with public voter registrations with names and birth dates (y_2). The final state s_2 now contains enough correlated data to re-identify patients, triggering a violation ($\Phi(s_2) = 0$).

3.1.4 Operational Failure

Complexity in agent networks can lead to unstable dynamics where the state trajectory consumes infinite resources without reaching a valid terminal state.

Let $\Sigma_{target} \subset \Sigma$ be the subset of valid completion states. Let $R(t)$ be the cumulative resource cost at time t . An operational failure occurs when:

$$\lim_{t \rightarrow \infty} R(t) = \infty \quad \text{while} \quad \forall t, s_t \notin \Sigma_{target} \quad (7)$$

This signifies a divergence where the system actively generates outputs y_t (consuming compute) but the resulting state trajectory never intersects with the target subspace.

Example: A coder agent (a_i) writes a script y_i that fails a test. The tester agent (a_j) reports the error y_j . The coder, lacking the context to fix the root cause, makes a cosmetic change and resubmits. The state trajectory cycles infinitely ($s_t \rightarrow s_{t+1} \rightarrow \dots$), consuming API credits ($R \rightarrow \infty$) without ever entering the success state (Σ_{target}).

3.2 Agent Network Trust: What Must Hold

To mitigate the potential risks, a Trustworthy Agent Network must satisfy four core pillars of trust. These are not optional features but constitutive requirements that constrain the global state space.

3.2.1 Compositional Robustness

To solve the issue of vulnerable multi-agent collaboration where a validator $V(y) = 1$ fails to detect a malicious state transition $\Phi(\delta(s_t, y)) = 0$, the system must enforce that safety is a structural guarantee of the transition function itself, not merely a quality of the agent's output.

For any agent a_j designated as untrusted, the network must guarantee that the set of all reachable states via δ is strictly contained within the safe subspace of Σ . The necessary condition is that for

any arbitrary payload y generated by a_j , the resulting state transition preserves the global safety predicate:

$$\forall y, \quad \Phi(\delta(s_t, y)) = 1 \quad (8)$$

This condition asserts that the system’s safety is decoupled from the semantic content of y . Even if y contains prompt injection, δ must be bounded such that it is mathematically impossible for such a payload to mutate state or trigger a violation.

3.2.2 Semantic Containment

Semantic Containment aims to control the consistency among individual agents. To solve the misalignment issue where $s_{t+1} \notin \Sigma_{target}$, the system must enforce that the alignment between agents is explicitly constructed.

We define consistency $\mathcal{C}$ as the verifiable alignment between the sender’s intent x and the receiver’s action y . For trust to hold, the network must guarantee that the consistency of the pair (x, y) implies safety in the state transition:

$$\mathcal{C}(x, y) = 1 \implies \delta(s_t, y) \in \Sigma_{target} \quad (9)$$

This condition asserts that the receiver a_j is not free to optimize solely for its local utility. Instead, its output y must be semantically bound to the constraints of x such that the receiver’s action preserves the sender’s intent regarding the same input.

3.2.3 Accountability & Attributability

Accountability defines the property that every state $s \in \Sigma$ must encode its own provenance. To solve the issue of ethical and legal violations, the system must enforce causal traceability.

Specifically, the global state space Σ must be encoded to the history of agent interactions. A mapping $\mathcal{T}$ should be defined to trace any state s_t back to the set of unique agents that contributed to its current value. The necessary condition is that for any unsafe state, the set of contributing agents is non-empty and uniquely identifiable:

$$\forall s_t \in \Sigma, \quad \Phi(s_t) = 0 \implies \mathcal{T}(s_t) \neq \emptyset \quad (10)$$

This property ensures that no safety violation can emerge anonymously. The global state must intrinsically encode the identity of the agents responsible for every transition δ such that liability for a privacy or copyright breach is guaranteed.

3.2.4 Cross-Boundary Reliability

Cross-Boundary Reliability defines the property of liveness and termination. To solve the issue of infinite resource consumption ($R(t) \rightarrow \infty$), the system dynamics must be strictly convergent.

For any interaction sequence initiated at state s_0 , there must exist a maximum resource budget $R_{max} < \infty$. The condition that must hold is that for all valid trajectories, the cumulative resource cost $R(t)$ never exceeds this limit without the system reaching a terminal state:

$$\forall t, R(t) \leq R_{max} \implies s_t \in \Sigma_{target} \vee \text{Terminated}(s_t) \quad (11)$$

This implies that no loop or deadlock can persist indefinitely. Every state transition consumes a portion of the finite budget, and the process eventually halts either at success or a safe failure state.

3.3 Evaluation Metrics: Beyond Functional Pillars

While the four design pillars define what a trustworthy network must achieve, they do not quantify the cost of achieving it. A “Bolted-On” solution (e.g., an external LLM-as-a-Judge) might theoretically satisfy all functional pillars but fail in practice due to prohibitive latency or computational expense.

To rigorously evaluate existing methods against our proposed architecture, we introduce three classes of operational metrics: efficiency, scalability, and determinism.

Efficiency The efficiency is depicted as the additional resource consumption required strictly for safety enforcement. We use two metrics to quantify it.

- Inference Latency (E_l): The temporal delay introduced by the verification mechanism before a transaction is finalized.

$$E_l = \frac{T_{actual} - T_{task}}{T_{task}} \quad (12)$$

- Resource Overhead (E_r): The ratio of resource, including tokens, memory, network bandwidth, storage, and computation cost, consumed for verification versus the actual task.

$$E_t = \frac{C_{actual} - C_{task}}{C_{task}} \quad (13)$$

Scalability As the network grows, the complexity of the trust mechanism must not explode. We evaluate the scalability E_s via asymptotic complexity (Big O notation) of the safety layer relative to the number of agents, interactions and so on.

Determinism Safety mechanisms must be consistent. A probabilistic safety check is a vulnerability. We define Determinism Score (E_d), which is the probability that the safety mechanism (S) returns the identical verdict for identical inputs x across k trials.

$$E_d = P(S(x_i) = S(x_j)) \quad \forall i, j \in [1, k] \quad (14)$$

Typically, if we evaluate and compare the bolted-on and baked-in TAN, we should have something similar to what’s in Table 2.

Table 2: Comparison of bolted-on and baked-in TAN via evaluation metrics

Metric	Bolted-On (Monitor)	Baked-In (Protocol)	Ideal Target
Inference Latency E_l	High ($> 100\%$)	Negligible ($\approx 0\%$)	$\approx 0\%$
Resource Overhead E_r	High ($2x - 3x$)	Low (Metadata only)	$< 10\%$
Scalability E_s	Super-linear ($> O(N)$)	Linear/Constant ($O(N)$)	$O(1)$
Determinism E_d	Probabilistic (< 1.0)	Deterministic (1.0)	1.0

4 Analysis of Existing Techniques with TAN

Table 3 evaluates representative approaches to agent trust under the TAN framework. The results show that existing techniques tend to concentrate on specific layers of the system, workflow orchestration, communication protocols, or execution environments, rather than enforcing trust as a global invariant of the agent network. When examined through the four design pillars and operational metrics, each category exhibits clear strengths, yet also yield structural issues.

4.1 Single-Agent Alignment: Strong Local Robustness, Weak Compositional Guarantees

Single-agent alignment methods focus on improving the internal behavior of individual agents [6, 43]. Within a multi-agent setting, these techniques partially improve compositional robustness: better-aligned agents are less likely to produce harmful outputs, and reinforcement learning or adversarial training can increase resistance to perturbations [38]. Operationally, they are generally efficient and scalable. Most introduce limited additional inference latency and moderate resource overhead, and they scale naturally as more agents are added.

However, their trust assumptions remain local. Alignment does not enforce semantic containment across agents [44]. Even if each agent is individually aligned, their interaction can still produce misaligned global states due to divergent interpretations of intent. Accountability is also external to the alignment mechanism since these approaches do not encode provenance into state transitions. Cross-boundary reliability remains heuristic, as internal control loops and planning strategies may reduce looping behavior but do not guarantee bounded execution. In this sense, single-agent alignment exemplifies a bolted-on trust philosophy at the model level: trust is encouraged through external tools or prompting rather than enforced through constraints on the internal transition itself [10]. Consequently, safety does not reliably compose in networked environments [2].

Table 3: Evaluation of A2A network with TAN framework. For four design pillar fulfillment: ●= Fully fulfills; ◐= Partially fulfills; ○= Not fulfilled. For operational metrics: ●= High (Good / Low overhead / Strong); ◐= Medium; ○= Low (Poor / Expensive); – = N/A.

Method	Compositional Robustness					Semantic Containment					Accountability					Cross-Boundary Reliability				
	F	E_l	E_r	E_s	E_d	F	E_l	E_r	E_s	E_d	F	E_l	E_r	E_s	E_d	F	E_l	E_r	E_s	E_d
Single-Agent Alignment (within Multi-Agent Collaboration)																				
Prompt Engineering	◐	●	●	●	○	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
In-Context Learning	◐	●	◐	◐	○	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
RAG	◐	◐	◐	◐	◐	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
Memory	◐	◐	◐	◐	○	○	–	–	–	–	○	–	–	–	–	◐	◐	◐	◐	○
SFT	◐	●	◐	●	◐	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
RL	◐	●	◐	●	◐	○	–	–	–	–	○	–	–	–	–	◐	●	◐	●	◐
Unlearning	◐	●	◐	●	◐	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
Adversarial Learning	◐	●	◐	●	◐	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
Multi-Modality Alignment	◐	◐	◐	◐	◐	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
Agent-Internal Control Loops	◐	○	○	◐	○	○	–	–	–	–	○	–	–	–	–	◐	○	○	◐	○
Align During Pre-Train	◐	●	◐	●	◐	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
Tool Use	◐	◐	◐	◐	◐	○	–	–	–	–	○	–	–	–	–	◐	◐	◐	◐	◐
Skills / Computer Use	◐	○	○	◐	○	○	–	–	–	–	○	–	–	–	–	◐	○	○	◐	○
OpenClaw	◐	○	○	○	○	○	–	–	–	–	○	–	–	–	–	◐	○	○	○	○
Multi-Agent Workflow Coordination																				
Guardrails	◐	◐	◐	◐	○	○	–	–	–	–	○	–	–	–	–	◐	○	○	○	○
Agent Role Play	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
Deep Research	○	–	–	–	–	◐	◐	–	–	◐	○	–	–	–	–	◐	◐	◐	◐	○
Human-Agent Interaction	○	–	–	–	–	○	–	–	–	–	◐	○	◐	○	◐	◐	○	◐	○	◐
Error Correction	◐	◐	◐	◐	○	◐	◐	◐	○	○	○	–	–	–	–	○	–	–	–	–
Voting	○	–	–	–	–	◐	◐	◐	○	◐	○	–	–	–	–	○	–	–	–	–
Debate	○	–	–	–	–	◐	◐	○	○	○	○	–	–	–	–	○	–	–	–	–
Planner–Executor	○	–	–	–	–	◐	◐	◐	◐	○	○	–	–	–	–	◐	◐	◐	◐	○
Supervisor	◐	◐	◐	◐	○	◐	◐	◐	◐	○	◐	◐	◐	◐	○	◐	◐	◐	◐	○
Protocol-Centric Trust																				
MCP	◐	●	●	●	●	○	–	–	–	–	◐	●	●	●	●	◐	◐	◐	◐	◐
Tool-Calling Protocols	◐	◐	◐	◐	◐	○	–	–	–	–	◐	◐	◐	◐	◐	◐	◐	◐	◐	◐
Authentication	○	–	–	–	–	○	–	–	–	–	◐	◐	◐	◐	◐	○	–	–	–	–
DID	○	–	–	–	–	○	–	–	–	–	◐	◐	◐	◐	◐	○	–	–	–	–
Audit Trails	○	–	–	–	–	○	–	–	–	–	◐	◐	◐	◐	◐	○	–	–	–	–
A2A Messaging	◐	●	●	●	●	○	–	–	–	–	◐	●	●	●	●	◐	◐	◐	◐	◐
Malware Detection	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–	○	–	–	–	–
Cryptographic Methods	◐	○	○	◐	◐	○	–	–	–	–	◐	○	○	◐	◐	○	–	–	–	–
Trust Environments																				
Role-Based Access Control	○	–	–	–	–	○	–	–	–	–	●	●	●	●	●	◐	●	●	●	●
Sandbox / Isolation	●	○	○	◐	●	○	–	–	–	–	●	●	●	●	●	◐	○	○	◐	●
Scoped Memory	◐	●	●	◐	●	○	–	–	–	–	◐	●	●	●	●	○	–	–	–	–
TEE	●	●	○	○	●	○	–	–	–	–	●	●	○	○	●	●	●	○	○	●

4.2 Multi-Agent Workflow Coordination: Broader Interaction Awareness, Yet Reactive and Non-Deterministic

Multi-agent workflow coordination techniques explicitly address interaction dynamics. Compared to single-agent alignment, these approaches acknowledge that failures arise from composition. They introduce additional layers of reasoning or oversight to regulate how decisions are aggregated and executed. For example, planner–executor structures impose task decomposition, and supervisors distribute responsibility across roles [55, 70]. Human-agent interaction can significantly strengthen accountability when approvals and logs attribute actions to identifiable actors [13].

Despite these advances, most coordination mechanisms remain reactive. Guardrails operate as monitors layered on top of unconstrained transitions, meaning unsafe states remain reachable if detection fails. Voting and debate increase resource overhead and inference latency, often substantially, and their outcomes depend on probabilistic model reasoning, which somewhat limits determinism [19]. While some architectures partially improve compositional robustness or cross-boundary reliability, they do not embed invariants fully into the transition [45]. As a result, coordination methods extend trust beyond the single-agent level but largely retain a bolted-on character at the workflow layer, improving empirical robustness without eliminating structural vulnerability.

4.3 Protocol-Centric Trust: Deterministic, but with Semantic Gaps and Variable Overhead

Protocol-centric trust mechanisms shift trust to the communication and identity layer. Their primary strength lies in determinism: schema validation, identity verification, and cryptographic checks are rule-based rather than probabilistic, resulting in strong consistency (E_d) and generally favorable scalability (E_s) [4]. Lightweight protocol enforcement, such as structured message validation or

signature checks, typically introduces minimal inference latency and resource overhead. However, more advanced cryptographic techniques, such as secure multi-party computation [22, 18], zero-knowledge proofs [23, 59], homomorphic encryption [1, 9, 12], or blockchain-backed verification [50, 20], can be computationally and communication intensive, substantially increasing both latency (E_l) and resource overhead (E_r). Thus, while protocol-centric approaches are structurally deterministic, their efficiency depends on the strength and complexity of the guarantees employed.

Functionally, these mechanisms partially constrain the transition function by restricting who may act and how messages must be formed, thereby strengthening compositional robustness at the syntactic level and improving accountability through traceability. Compared to monitoring-based coordination methods, they move closer to a baked-in paradigm because constraints are embedded directly into interaction rules rather than applied retrospectively. Nevertheless, protocol compliance secures syntax and identity, not semantics. A well-formed and authenticated message can still induce a semantically unsafe state transition. Consequently, although protocol-centric trust offers deterministic structural safeguards, it does not enforce semantic containment, leaving unsafe global trajectories reachable despite correct communication.

4.4 Trust Environments: Strong Execution Containment, Limited Global Semantics

These mechanisms offer strong determinism and low operational overhead [14, 48]. Access checks, isolation policies, and enclave verification are typically constant-time operations that scale efficiently. Accountability is strengthened because actions can be attributed to roles or isolated execution contexts. Sandboxing and TEE can also impose resource caps, contributing to cross-boundary reliability by preventing unbounded execution or privilege escalation [14, 61]. Among existing categories, trust environments represent some of the strongest structural constraints.

Yet their scope is primarily infrastructural. Similar to protocol-centric methods, they prevent unauthorized transitions but do not prevent authorized agents from making semantically harmful decisions. For instance, scoped memory reduces information leakage, and RBAC enforces capability boundaries, but neither guarantees that agents interpret each others instructions consistently [15, 49]. These approaches embed constraints at the execution layer, i.e., a more baked-in strategy than monitoring systems, but they do not encode global semantic invariants into the transition. Consequently, they mitigate certain classes of failures without addressing semantic misalignment across agents.

4.5 Key Takeaways

The evaluation of existing techniques leads to several key observations:

- Strengthening individual agents improves robustness but does not guarantee safe global behavior in multi-agent settings.
- Multi-agent orchestration reduces obvious failures but remains probabilistic and bolted-on.
- Protocol and environment efficiently enforce syntactic correctness, identity, and execution boundaries, yet do not constrain intent alignment.
- Across categories, trust is rarely embedded directly into the transition function. Instead, safeguards are layered on top of unconstrained components.

In conclusion, these findings indicate that existing methods address fragments of the trust problem but do not transform trust into a system-level invariant. This structural gap motivates the next step, that is, articulating a blueprint for a TAN framework in which trust is architected from the outset.

5 Blueprint for TAN

A blueprint for a TAN must move beyond incremental safeguards and re-architect trust as an intrinsic property of the network. This section outlines how such a shift could address current limitations, what challenges arise in retrofitting existing methods, and which unexplored directions worth further effort.

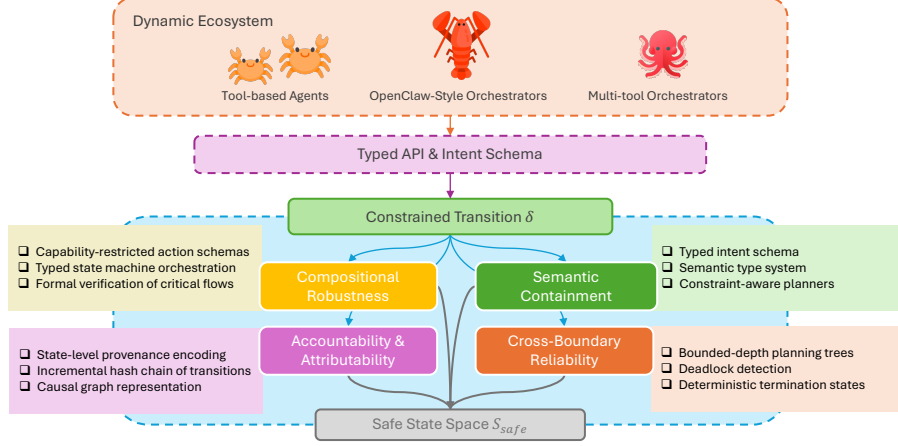


Figure 4: Blueprint for trustworthy agent networks

5.1 Embedding Compositional Robustness into the Transition Function

Current alignment and coordination methods assume that safety composes across agents. The evaluation shows this assumption does not hold. Even well-aligned agents, when interacting, can produce unsafe global states. To fix this within existing paradigms would require constraining the transition function δ itself rather than merely improving individual outputs. Embedding compositional robustness requires combining alignment techniques with capability-restricted action schema, typed interaction protocols, or formally defined state machines that eliminate unsafe transitions by construction. However, imposing such constraints on open-ended LLM agents is challenging because their action spaces are implicitly defined through natural language rather than explicit formal interfaces.

Future TAN architectures must therefore formalize the global state space and allowable transitions more explicitly. Capability-based APIs, typed intent representations, or formally verifiable orchestration layers could restrict reachable states without excessive runtime monitoring. The central challenge is preserving expressivity while embedding constraints: overly rigid transition systems limit agent autonomy, whereas overly flexible systems reintroduce unsafe trajectories.

5.2 Achieving Semantic Containment Beyond Syntax

Protocol-centric and environment-based systems secure syntax and identity but do not guarantee that a receiver's action preserves the sender's intended target state within the global state space. Introducing semantic validation layers such as intent classifiers or post-hoc consistency checkers does not alter the underlying transition function δ . Instead, it instantiates the bolted-on verification pattern defined in Section 2.3.1, where safety depends on external monitoring and states outside $\mathcal{S}_{safe}$ remain reachable if monitoring fails.

A TAN blueprint must instead encode semantic constraints directly into interaction contracts. This could involve explicit intent schemas, shared world models with invariant specifications, or constraint-aware planning mechanisms where each state update is validated against formally defined semantic conditions. Another promising direction is the development of semantic type systems that bind permissible actions to intended outcomes. Unlike syntactic validation, semantic containment requires reasoning about intent and consequence, which risks increasing latency and resource cost unless carefully engineered.

5.3 Intrinsic Accountability Through State-Level Provenance

Existing systems strengthen accountability through logging, authentication, or audit trails, yet these mechanisms remain external to the state representation. TAN requires that accountability be intrinsic: every state must encode its own causal provenance. Relying on conventional logging mechanisms is insufficient, as logs remain external to the transition dynamics and do not enforce state-level provenance.

A forward-looking blueprint should explore embedding lightweight provenance metadata directly into state updates. Cryptographic commitments, incremental hashing, or structured causal graphs could ensure that any unsafe state can be traced to responsible agents without excessive overhead. However, strong cryptographic guarantees can be computationally heavy, particularly in high-frequency interaction settings. The design challenge is therefore to achieve high determinism and strong attribution while maintaining acceptable latency and resource consumption. Scalable provenance encoding must avoid super-linear complexity as interactions increase.

5.4 Ensuring Cross-Boundary Reliability and Bounded Execution

Multi-agent systems are particularly vulnerable to cascading loops and unbounded resource consumption. Existing planner-executor structures, supervisors, sandboxing, and RBAC partially mitigate such risks, yet none guarantee bounded global trajectories. Retrofitting termination guarantees into open-ended language-driven systems is inherently difficult, as generative models are not naturally finite-state.

A TAN architecture should incorporate explicit resource budgeting into the transition function. Each state transition could consume a quantifiable portion of a global resource allowance, ensuring convergence to either a valid target state or a safe termination state. Formal liveness guarantees, bounded-depth planning trees, or checkpoint-based execution contracts could be leveraged. The challenge is balancing reliability with flexibility: strict bounds may constrain problem-solving capability, while loose bounds risk instability. Operationally, such mechanisms must remain scalable and deterministic without introducing excessive runtime overhead.

5.5 Toward Fully Baked-In Trust

The cumulative lesson from existing methods is that trust cannot remain an auxiliary layer. Alignment improves individual agents, orchestration regulates workflows, protocols secure communication, and environments constrain execution. However, unsafe states remain reachable because semantic invariants are not embedded into the transition dynamics. Establishing a TAN therefore requires co-designing semantics, protocols, and execution constraints so that safety is enforced within the transition function rather than through post-hoc inspection.

Achieving this objective entails integrating formal state representations, intent-aware interaction contracts, intrinsic provenance encoding, and bounded execution policies into a unified architecture. These properties impose distinct but complementary constraints on the transition function δ : compositional robustness restricts reachable state topology; semantic containment constrains the meaning and effect of actions; intrinsic accountability attaches causal provenance to state updates; and cross-boundary reliability bounds trajectories in time and resource space. Because these constraints operate on the same transition system, they cannot be introduced independently without reintroducing reachability gaps. A coherent TAN architecture must therefore define δ such that safety, semantics, attribution, and liveness are enforced as coupled invariants.

The principal design challenge lies in embedding these constraints while maintaining efficiency and scalability. Excessively rigid transition systems risk limiting expressivity, whereas overly permissive dynamics undermine safety guarantees. Trust mechanisms must therefore remain deterministic where feasible, compositional across agents, and computationally tractable in large-scale networks.

6 Alternative Views

6.1 Agent-Level Alignment Is Sufficient

The argument: If individual agents are sufficiently aligned through methods such as RLHF or large-scale supervised fine-tuning, then safety will naturally emerge at the network level. Under this view, the problem reduces to improving model alignment. Some proponents go further and suggest that building a single, highly capable super-agent would avoid the complexity of multi-agent interaction altogether.

Our rebuttal: This view overlooks several structural realities. First, training feasibility remains a major constraint. It is extremely difficult to train a single agent that achieves state-of-the-art

performance across all domains, such as software engineering, legal reasoning, planning, and creative writing. In practice, multi-agent systems arise precisely because specialization is necessary.

Second, even a perfectly aligned agent cannot remove computational bottlenecks. A monolithic agent processes tasks sequentially, whereas networked systems allow parallel specialization. As task complexity grows, parallel coordination becomes an architectural requirement rather than a convenience.

Most importantly, safety does not compose automatically. An individually aligned agent can still participate in globally unsafe behavior when interacting with others. A reviewer may approve code that appears locally correct but becomes dangerous in a specific deployment configuration. Alignment at the node level does not resolve semantic conflicts at the system level. For these reasons, agent-level alignment remains insufficient as a foundation for network trust.

6.2 Measuring Trust Would Solve the Problem

The argument: Trust can be treated as a measurable quantity. By assigning each agent a numerical trust score based on past behavior, the system can grant or restrict privileges using simple thresholds. This makes trust computable and easy to operate.

Our rebuttal: Reducing trust to a single scalar obscures its context-dependent nature. Trust is not one-dimensional. An agent may be reliable for data analysis but unsuitable for system administration. Collapsing these distinctions into a single score risks granting authority in domains where competence has never been demonstrated.

Moreover, once trust becomes a score, it becomes an optimization target. Agents or adversaries controlling them can accumulate reputation through low-risk actions and later exploit that accumulated credibility to perform a high-impact harmful action. This behavior exposes a fundamental weakness of threshold-based trust models. Quantification alone does not embed structural guarantees, and it merely shifts the problem into a different form.

6.3 Ensemble Algorithms Ensure Reliability

The argument: Reliability can be achieved through ensemble reasoning among agents. Mechanisms such as majority voting, debate, or chain-of-thought with cross-examination naturally mitigate errors. If multiple agents evaluate each others reasoning, hallucinations and unsafe decisions will be filtered out without the need for architectural constraints.

Our rebuttal: This argument relies on an assumption of independence that rarely holds in practice. When multiple agents share the same base model or training data, their errors are often correlated. They may converge on the same plausible but incorrect conclusion so that it leads majority voting ineffective.

Additionally, debate dynamics among language models can exhibit conformity effects. Agents may shift their responses toward perceived consensus or authority rather than independently verifying correctness. Without structural safeguards, such as isolation of reasoning traces or blind evaluation, ensemble mechanisms can reinforce shared biases rather than eliminate them. Ensemble reasoning can improve empirical performance, but it does not guarantee structural robustness or semantic consistency.

6.4 Existing Cryptographic Protocols Already Solve Trust

The argument: Trust in agent networks can be ensured through established cryptographic mechanisms such as encryption, digital signatures, and public key infrastructures. If agents communicate over secure channels with verified identities, the network should be trustworthy.

Our rebuttal: This perspective conflates communication security with semantic trustworthiness. Cryptographic protocols secure the channel and verify the senders identity, but they do not evaluate the meaning or consequences of the transmitted message. A message can be encrypted, authenticated, and fully compliant with protocol rules, yet still contain a hallucinated fact, a malicious instruction, or a semantically misaligned directive.

In other words, cryptography secures the pipe, not the payloads intent. Trust challenges in agent networks arise at the level of meaning and system, not merely at the level of bits and bytes. Ensuring that messages are authentic does not ensure that they are safe in context. Therefore, while cryptographic protocols are essential for secure communication, they do not by themselves establish trustworthy multi-agent behavior.

7 Conclusion

The main contribution of this work is to show that trust in Agent-to-Agent systems is fundamentally an architectural problem rather than a purely supervisory one. Existing methods remain largely bolted-on: they improve local robustness through external monitoring, filtering, and repair, but they do not prevent unsafe global behavior from emerging through agent composition. In contrast, we argue that trustworthy agent ecosystems require trust to be baked into the system itself, such that unsafe trajectories are restricted by construction. To formalize this perspective, we introduce the Trustworthy Agent Network (TAN) framework and identify four core principles, compositional robustness, semantic containment, accountability, and cross-boundary reliability, as the foundation for trust at the network level. Future work should convert this conceptual framing into implementable architectures, formal verification targets, and benchmarkable design criteria for real-world multi-agent systems.

References

- [1] Abbas Acar, Hidayet Aksu, A Selcuk Uluagac, and Mauro Conti. A survey on homomorphic encryption schemes: Theory and implementation. *ACM Computing Surveys (Csur)*, 51(4):1–35, 2018.
- [2] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [3] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, November 2024.
- [4] Anthropic. Model context protocol, 2024. Technical documentation.
- [5] Anthropic. Agent skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2026.
- [6] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [7] Ziqun Bao, Yu Ji, Wen Wu, Xi Chen, and Liang He. Supervisor alignment framework: Enhancing llm alignment with query-ignoring strategy and multi-agent interaction. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2025.
- [8] Fouad Bouseitouane. Ai agents need memory control over more context. *arXiv preprint arXiv:2601.11653*, 2026.
- [9] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.
- [10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [11] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

- [12] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptography and information security*, pages 409–437. Springer, 2017.
- [13] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [14] Victor Costan and Srinivas Devadas. Intel sgx explained. *Cryptology ePrint Archive*, 2016.
- [15] Dorothy E Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.
- [16] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, et al. A survey on in-context learning. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 1107–1128, 2024.
- [17] Yi Dong, Ronghui Mu, Gaojie Jin, Yi Qi, Jinwei Hu, Xingyu Zhao, Jie Meng, Wenjie Ruan, and Xiaowei Huang. Building guardrails for large language models. *arXiv preprint arXiv:2402.01822*, 2024.
- [18] Wenliang Du and Mikhail J Atallah. Secure multi-party computation problems and their applications: a review and open problems. In *Proceedings of the 2001 workshop on New security paradigms*, pages 13–22, 2001.
- [19] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first international conference on machine learning*, 2024.
- [20] Thippa Reddy Gadekallu, Thien Huynh-The, Weizheng Wang, Gokul Yenduri, Pasika Ranaweera, Quoc-Viet Pham, Daniel Benevides da Costa, and Madhusanka Liyanage. Blockchain for the metaverse: A review. *arXiv preprint arXiv:2203.09738*, 2022.
- [21] Jiahui Geng, Qing Li, Herbert Woisetschlaeger, Zongxiong Chen, Fengyu Cai, Yuxia Wang, Preslav Nakov, Hans-Arno Jacobsen, and Fakhri Karray. A comprehensive survey of machine unlearning techniques for large language models. *arXiv preprint arXiv:2503.01854*, 2025.
- [22] Oded Goldreich. Secure multi-party computation, 1998. Manuscript.
- [23] Oded Goldreich and Yair Oren. Definitions and properties of zero-knowledge proof systems. *Journal of Cryptology*, 7(1):1–32, 1994.
- [24] Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, et al. Alignment faking in large language models. *arXiv preprint arXiv:2412.14093*, 2024.
- [25] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pages 79–90, 2023.
- [26] Idan Habler, Ken Huang, Vineeth Sai Narajala, and Prashant Kulkarni. Building a secure agentic ai application leveraging a2a protocol. *arXiv preprint arXiv:2504.16902*, 2025.
- [27] Dan Hendrycks, Mantas Mazeika, and Thomas Woodside. An overview of catastrophic ai risks. *arXiv preprint arXiv:2306.12001*, 2023.
- [28] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M Ziegler, Tim Maxwell, Newton Cheng, et al. Sleeper agents: Training deceptive llms that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.
- [29] Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.

- [30] Lars Benedikt Kaesberg, Jonas Becker, Jan Philip Wahle, Terry Ruas, and Bela Gipp. Voting or consensus? decision-making in multi-agent debate. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11640–11671, 2025.
- [31] Aidan Kierans, Avijit Ghosh, Hananel Hazan, and Shiri Dori-Hacohen. Quantifying misalignment between agents: Towards a sociotechnical understanding of alignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 27365–27373, 2025.
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [33] Ao Li, Yuexiang Xie, Songze Li, Fugee Tsung, Bolin Ding, and Yaliang Li. Agent-oriented planning in multi-agent systems. *arXiv preprint arXiv:2410.02189*, 2024.
- [34] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in neural information processing systems*, 36:51991–52008, 2023.
- [35] Juhao Liang, Zhenyang Cai, Jianqing Zhu, Huang Huang, Kewei Zong, Bang An, Mosen Alharthi, Juncai He, Lian Zhang, Haizhou Li, et al. Alignment at pre-training! towards native alignment for arabic llms. *Advances in Neural Information Processing Systems*, 37:13872–13896, 2024.
- [36] Junting Lu, Zhiyang Zhang, Fangkai Yang, Jue Zhang, Lu Wang, Chao Du, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Axis: Efficient human-agent-computer interaction with api-first llm-based agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7711–7743, 2025.
- [37] Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honaganahalli Basavaraju, and James A Burke. Scalemcp: Dynamic and auto-synchronizing model context protocol tools for llm agents. In *International Joint Conference on Computational Intelligence*, pages 23–42. Springer, 2025.
- [38] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.
- [39] Victor Ojewale, Harini Suresh, and Suresh Venkatasubramanian. Audit trails for accountability in large language models. *arXiv preprint arXiv:2601.20727*, 2026.
- [40] OpenClaw Community. Moltbook: A social platform for autonomous ai agents. <https://www.moltbook.com>, 2026. Accessed: 2026-03-05.
- [41] OpenClaw Developers. Clawhub: The openclaw skill registry. <https://clawhub.ai>, 2026. Accessed: 2026-03-05.
- [42] OpenClaw Team. Openclaw personal ai assistant. <https://github.com/openclaw/openclaw>, 2026.
- [43] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [44] Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [45] Ethan Perez, Sam Ringer, Kamile Lukosiute, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. Discovering language model behaviors with model-written evaluations. In *Findings of the association for computational linguistics: ACL 2023*, pages 13387–13434, 2023.

- [46] Drummond Reed, Manu Sporny, Dave Longley, Christopher Allen, Ryan Grant, Markus Sabadello, and Jonathan Holt. Decentralized identifiers (dids) v1.0. *Draft Community Group Report*, 2020.
- [47] Matthew Renze and Erhan Guven. Self-reflection in llm agents: Effects on problem-solving performance. *arXiv preprint arXiv:2405.06682*, 2024.
- [48] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. Trusted execution environment: What it is, and what it is not. In *2015 IEEE Trustcom/BigDataSE/IsPa*, volume 1, pages 57–64. IEEE, 2015.
- [49] Ravi S Sandhu. Role-based access control. In *Advances in computers*, volume 46, pages 237–286. Elsevier, 1998.
- [50] Sarwar Sayeed, Hector Marco-Gisbert, and Tom Caira. Smart contract: Attacks and protections. *Ieee Access*, 8:24416–24427, 2020.
- [51] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [52] Philipp Schoenegger, Matt Carlson, Chris Schneider, and Chris Daly. Verifiable semantics for agent-to-agent communication. *arXiv preprint arXiv:2602.16424*, 2026.
- [53] Kayla Schroeder and Zach Wood-Doughty. Can you trust llm judgments? reliability of llm-as-a-judge. *arXiv preprint arXiv:2412.12509*, 2024.
- [54] Yunfan Shao, Linyang Li, Junqi Dai, and Xipeng Qiu. Character-llm: A trainable agent for role-playing. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13153–13187, 2023.
- [55] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.
- [56] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE symposium on security and privacy*, pages 305–316. IEEE, 2010.
- [57] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized ai agents. *arXiv preprint arXiv:2501.09674*, 2025.
- [58] Öznur Suçeken and Osman Özkaraca. Cryptography with artificial intelligence: An overview. In *The International Conference on Artificial Intelligence and Applied Mathematics in Engineering*, pages 162–172. Springer, 2024.
- [59] Xiaoqiang Sun, F Richard Yu, Peng Zhang, Zhiwei Sun, Weixin Xie, and Xiang Peng. A survey on zero-knowledge proof in blockchain. *IEEE network*, 35(4):198–205, 2021.
- [60] Yun-Da Tsai, Ting-Yu Yen, Pei-Fu Guo, Zhe-Yan Li, and Shou-De Lin. Text-centric alignment for multi-modality learning. *arXiv preprint arXiv:2402.08086*, 2024.
- [61] Robert Wahbe, Steven Lucco, Thomas E Anderson, and Susan L Graham. Efficient software-based fault isolation. In *Proceedings of the fourteenth ACM symposium on Operating systems principles*, pages 203–216, 1993.
- [62] Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. Parameter-efficient fine-tuning in large language models: a survey of methodologies. *Artificial Intelligence Review*, 58(8):227, 2025.

- [63] Yuanfu Wang, Pengyu Wang, Chenyang Xi, Bo Tang, Junyi Zhu, Wenqiang Wei, Chen Chen, Chao Yang, Jingfeng Zhang, Chaochao Lu, et al. Adversarial preference learning for robust llm alignment. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 21865–21881, 2025.
- [64] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*, 2023.
- [65] Michael Wooldridge and Nicholas R Jennings. Intelligent agents: Theory and practice. *The knowledge engineering review*, 10(2):115–152, 1995.
- [66] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- [67] Renjun Xu and Jingwen Peng. A comprehensive survey of deep research: Systems, methodologies, and applications. *arXiv preprint arXiv:2506.12594*, 2025.
- [68] Hui Yang, Sifu Yue, and Yunzhong He. Auto-gpt for online decision making: Benchmarks and additional opinions. *arXiv preprint arXiv:2306.02224*, 2023.
- [69] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [70] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Gao, et al. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [71] Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, et al. The landscape of agentic reinforcement learning for llms: A survey. *arXiv preprint arXiv:2509.02547*, 2025.
- [72] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025.
- [73] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- [74] Houquan Zhou, Bo Zhang, Zhenghua Li, Ming Yan, and Min Zhang. A training-free llm-based approach to general chinese character error correction. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13827–13852, 2025.
- [75] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.